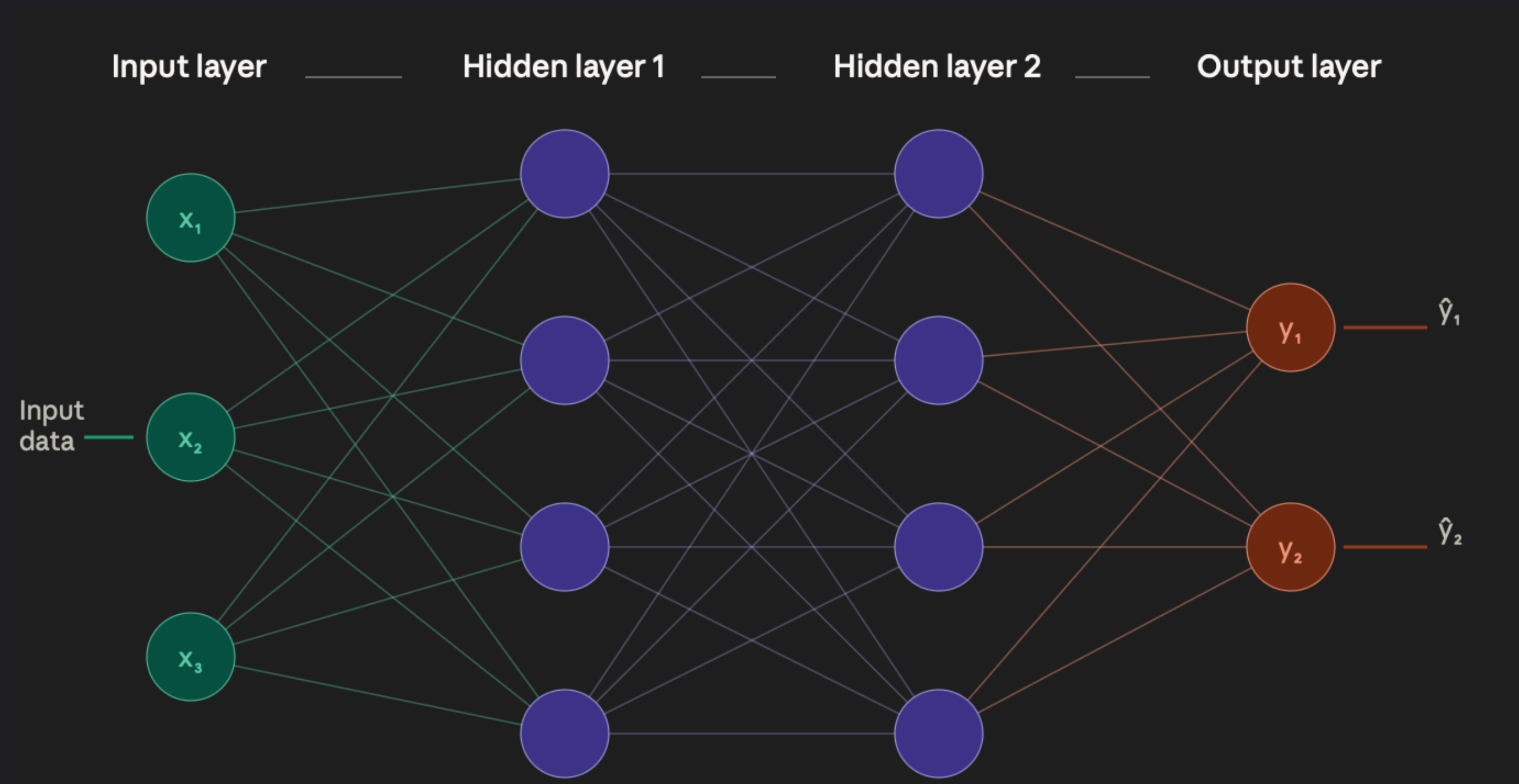


What is a Neural Network

A neural network is a system of connected nodes organized in layers that learns patterns from data. The inspiration comes from how the brain works: signals travel from neuron to neuron, getting transformed at each step until an answer comes out.



A Neural Network (Architectural Diagram)

Layers

A layer is a group of neurons that process data at the same stage. Every neural network has three:

1. The Input Layer takes in raw data,
2. The Hidden Layers transform and learn from it,
3. The Output Layer produces the final result.

The more hidden layers you stack, the more complex the patterns the network can pick up. That depth is literally where the term "deep learning" comes from.

Neurons

A neuron receives inputs, multiplies each by a weight, sums them up, and pushes the result through an activation function before passing it along. Think of it like a filter: it decides how strongly to forward a signal based on what it has learned.

Weights

A weight is a number attached to each connection between neurons. It controls how much one neuron influences the next. High weight means strong influence. Low or negative weight means weak or opposing influence. When you train a neural network, you are really just adjusting these weights. The network makes a wrong prediction, the weights shift slightly, and it does better next round. That is the whole learning process.

Activation Functions

An activation function decides whether a neuron fires and in what form. Without one, the network can only learn straight-line relationships. ReLU passes positive values and blocks negatives, which is why it shows up in most hidden layers. Sigmoid squishes output between 0 and 1, useful for yes/no predictions. Softmax turns outputs into probabilities across multiple categories. I think of it as a gate on each neuron: the function controls how open or closed that gate is.

Loss Function

A loss function measures how wrong a prediction is. High loss means the network is far off. Low loss means it is close. Mean Squared Error works when you are predicting a number, like house prices. Cross-Entropy Loss fits when you are classifying categories, like cat versus dog. Everything the network does is aimed at pushing this number down.

Optimization Algorithms

An optimization algorithm is how the network actually reduces its loss by adjusting weights after each prediction. Gradient Descent calculates the direction of the error and takes a small step toward fixing it. Adam Optimizer adapts the step size for each weight automatically, which makes it smarter in practice. The analogy I keep coming back to: you are blindfolded in a hilly landscape, trying to find the lowest point. Each step you take is guided by the slope under your feet.

Summary

Neural networks are not mysterious once you see the parts:

- Layers organize neurons into input, hidden, and output stages
- Neurons receive, process, and forward signals
- Weights set the strength of every connection
- Activation functions let the network learn non-linear patterns
- Loss functions score how wrong the predictions are
- Optimization algorithms fix the weights to reduce that score

Personal Reflection

What I found genuinely surprising working through this was how central weights are to everything. Every layer, every neuron, every prediction ultimately depends on whether those weights have been tuned correctly. The intelligence is not in any single component. It comes from millions of small, simple operations running together.

Working with TensorFlow Playground made this click in a way that reading about it did not. I could watch how adding hidden layers changed the decision boundary in real time. It stopped being abstract. That, I think, is the real test of whether you understand something: can you explain it simply enough that someone else could draw it?

Appreciation

Thank You for your
Attention